

Marathi Machine Translation via Bodhan Fine-Tune

AI4Bharat — AI Research Engineer take-home
English → Marathi (eng_Latn → mar_Deva)

Source code: <https://github.com/Kaustubh-Rathi/marathi-mt-bodhan>

Artifacts, checkpoints & logs: https://drive.google.com/drive/folders/1Abt7fldMDI_4pjTa3ayDbLI1dqDZSUTD

Primary model: bodhan-ai/indic-translate (Gemma-4 E4B, 8B), QLoRA

Train: ai4bharat/samanantar (config `mr`), 8,000 pairs

Eval (held out, never trained on): ai4bharat/IN22-Gen (test), facebook/flores eng_Latn-mar_Deva (devtest)

1. What was delivered

Three fine-tuning sessions, each run end-to-end on a Kaggle GPU, with a full resumable checkpoint saved and mirrored to Drive every 5 optimizer steps.

Session	Method	Steps	Status	GPU
A — primary: Bodhan 8B QLoRA	qlora	16		–
B — fallback: IndicTrans2 200M LoRA	lora-seq2seq	16		–
C — ablation (r 16/ α 32, LR 5e-5)	qlora	16		–

Session A is the assignment-aligned primary. Session B is an independent 200M fallback that guarantees a complete story regardless of what happens on the 8B. Session C is a reduced-adapter / lower-LR ablation of A. Configs are written for the full schedule; the submitted runs are 20-step smoke runs that prove the pipeline, so the numbers below are a lower bound rather than a converged result.

Evaluation (Session B, held-out sets)

Test set	BLEU	chrF2	chrF2++
IN22-Gen test (n=1024)	20.91	54.10	49.82
FLORES eng_Latn-mar_Deva devtest (n=1012)	19.49	54.90	50.69

2. Key decisions and why

Decision	Choice	Reasoning
Base model	<code>bodhan-ai/indic-translate</code> (8B) as primary; IndicTrans2 200M as fallback	The 8B is the assignment-aligned pick and the stronger Marathi prior, but it is gated and must fit a T4. The 200M trains anywhere and de-risks the schedule, so it runs in parallel rather than as a fallback branch.
Fine-tune stack	TRL + PEFT + BitsAndBytes NF4	Transparent and auditable. Unsloth is faster but its patched kernels hide exactly the failures that needed diagnosing; Axolotl adds a layer between me and those errors.
LoRA target modules	<code>all-linear</code> minus vision/audio and <code>lm_head</code>	Gemma-4 uses <code>ClippableLinear</code> ; a bare <code>q_proj</code> list crashes adapter injection. This form is encoded in <code>base.yaml</code> .
Experiment tracking	TensorBoard + <code>experiments.csv</code> + Drive sync	Zero infrastructure. A hosted tracker was rejected because it adds account and egress friction across three Kaggle accounts for three runs.
Compute	3 Kaggle accounts, one session each	A single account's 30h/week means one failure ends the story. Each session streams checkpoints to Drive, so a 12h kill costs at most one checkpoint.
Optimizer	<code>paged_adamw_8bit</code> , no DeepSpeed	Paged optimizer absorbs T4 memory spikes. ZeRO-3 shards the frozen base out from under PEFT and is wrong for single-GPU LoRA.

3. Data, evaluation and decontamination

Dataset pivot. The preferred education-domain set was access-gated and I was denied, so training uses the open `ai4bharat/samanantar` (`mr`) slice, capped at 8,000 train / 1,000 dev pairs, seed 42.

Evaluation is on general-domain sets disjoint from the training slice — IN22-Gen (test) and FLORES devtest — so any gain has to come from transfer rather than leakage.

Decontamination (full L1–L10 table in `reports/LEAKAGE_CHECKLIST.md`): pair-hash blocklist of the test sets against train, exact and source-only near-duplicate filtering at 0.9 similarity, 1–100 word length filters, and provenance checks. Scoring decoding is fixed at 256 new tokens, beam 5.

4. Engineering log: four failures, diagnosed not worked around

Each of these killed a run at the first optimizer step. All four are fixed in code, documented inline at the fix, and re-verified by a completed run.

1. **QLoRA OOM (10.5 GiB on a 16 GiB T4)**. PEFT's stock `prepare_model_for_kbit_training` upcasts every low-precision tensor to FP32, including Gemma's multi-gigabyte embedding table. Replaced with an architecture-aware helper that upcasts only 1-D norm vectors and keeps large matrices in the compute dtype.
2. **assistant_only_loss RuntimeError**. TRL aborts when a row has no assistant mask, because the gated chat template emits no generation markers. Disabled it for these runs and recorded the semantic consequence (full-sequence loss) rather than silently accepting it.
3. **GradScaler vs BF16**. On a T4, `torch.cuda.is_bf16_supported()` returns True through sm75 emulation, so the model loaded in BF16 while the trainer ran the FP16 scaler: `_amp_foreach_non_finite_check_and_unscale_cuda` not implemented for 'BFloat16'. Fixed by letting explicit config flags override the GPU capability probe instead of trusting the probe.
4. **FP16 master weights**. Forcing FP16 only moved the failure: `Attempting to unscale FP16 gradients`. The AMP contract needs FP32 master weights with autocast doing the casting, so trainable LoRA params are upcast to FP32 *after* the adapters are built — they are created inside `SFTTrainer.__init__`, so aligning earlier had missed all 100,999,168 of them. The frozen 4-bit base stays low-precision, keeping memory in budget.

5. Reproducing this

```
pip install -r requirements.txt
make data          # download + decontaminate + build train/dev/test
make train-a       # primary Bodhan 8B QLoRA
make train-b       # IndicTrans2 fallback
make eval          # IN22-Gen + FLORES -> reports/metrics.json
```

Training runs as Kaggle *script* kernels (no notebooks), one account per session, driven by `scripts/kaggle/push_kernel.ps1`. Every checkpoint is rcloned to Drive in the background as it is written, each verified with an `_upload_complete` marker before local copies are pruned; after a 12h kill, set `MR_MT_RESUME=auto` and re-push to resume from the last confirmed checkpoint.

6. Honest caveats

- Training data pivoted away from the preferred education set because access was denied, and Samanantar was part of IndicTrans2's pretraining corpus, so Session B's score partly re-measures memorisation.
- These are 20-step smoke runs, not converged fine-tunes. The metric table demonstrates the evaluation path works end to end.
- Assistant-only loss is disabled, so the loss covers the full sequence rather than the assistant span alone.
- Session B logs gradient norms around 3.5e4 against `max_grad_norm=1.0` — heavy clipping every step. A lower learning rate or longer warmup is the first thing to change for a real run.

Full detail lives in the repository: `APPROACH.md` (narrative and rejected alternatives), `docs/HYPERPARAMETERS.md` (every knob and its rationale), `docs/SPEC.md`, `docs/SETUP.md`, `docs/TRAINING.md`, and `reports/` for live artifacts.